Building a project with Claude Code

A study guide for the human in the loop, using the Spring microservices on Kubernetes (laptop) project as the worked example

What you do yourself, what you hand to Claude, every instruction component you give it, and how to run the project step by step (workflow) or with agents doing more of it (agentic).

Companion files in this repo: `docs/build-from-scratch-checklist.md` (the phase checklist) and `docs/templates/CLAUDE.md` (the CLAUDE.md to start a new repo with).

October 2026

Contents

1. How to read this guide 4
   - The project in one paragraph 4
2. Division of labour: human vs Claude 5
   - Things only a human can do in this project 5
3. The instruction stack: everything you give Claude 7
4. Each component in detail 8
   - 4.1 The prompt 8
   - 4.2 CLAUDE.md: project memory 8
   - 4.3 Docs and specs 9
   - 4.4 Permissions: settings.json 9
   - 4.5 Hooks: automatic guarantees 10
   - 4.6 Skills: reusable procedures 10
   - 4.7 Subagents: focused roles 11
   - 4.8 MCP servers: reach 11
   - 4.9 Auto memory and modes 11
5. Two approaches: workflow vs agentic 13
   - 5.1 Workflow (human-driven) 13
   - 5.2 Agentic (Claude drives within guardrails) 13
   - 5.3 Side by side 14
   - 5.4 Recommended: the ratchet 14
6. Phase playbook 16
   - Phase 0: Laptop prerequisites 16
   - Phase 1: Repository skeleton 16
   - Phase 2: Microservices (product → inventory → order → notification → gateway → frontend) 17
   - Phase 3: Local dev with docker compose (optional) 18
   - Phase 4: Containerisation 18
   - Phase 5: Harbor registry 19
   - Phase 6: Kubernetes manifests 19
   - Phase 7: First deploy (backend only) 20
   - Phase 8: GitOps with Argo CD 20
   - Phase 9: Release loop 21

Phase 10-11: Docs and agentic setup 21
7. Writing instructions that work 23
Patterns worth memorising 23
8. Challenges and how to handle them 24
9. Claude Code cheat sheet 25
10. Learning path: exercises 26
Appendix A: Starter files 27
A.1 CLAUDE.md for a from-scratch build (excerpt) 27
A.2 Subagent skeleton 27
A.3 Hook skeleton (PreToolUse, Bash) 27
A.4 Phase prompt template 28
Appendix B: Case study: how this guide's own session went 29

1. How to read this guide

This guide answers four questions, in order:

1. **What is my job, and what is Claude's?** (section 2) Some things only a human can or should do. Knowing that list is most of the skill.
2. **What do I actually give Claude?** (sections 3 and 4) A project is driven by more than prompts. There is an *instruction stack*: CLAUDE.md, docs, permissions, hooks, skills, subagents, MCP servers and memory. Each one is covered with what it is, where it lives, what to put in it and an example from this project.
3. **Workflow or agentic?** (section 5) Two ways to run the same project, their challenges, and the approach recommended here: start with the workflow, then turn what works into agentic tooling.
4. **What happens in each phase?** (section 6) A playbook for phases 0 to 11 of the checklist: what you do, the prompt you give, the gate that proves the phase works, and what usually goes wrong.

Sections 7 to 10 cover prompt writing, common challenges, a Claude Code cheat sheet and a set of exercises. The appendices hold copy-ready templates.

> **The one idea to keep**
>
> You are the **architect, decision-maker, approver and owner**. Claude is a fast, capable **implementer and reviewer** that knows only what you write down. Almost every problem in AI-assisted development comes from breaking that split: either you hand over a decision that was yours, or you keep doing typing that Claude could do.

The project in one paragraph

Five Spring Boot 3.2 / Java 21 services (product, order, inventory, notification, api-gateway) plus an Angular frontend. They use MongoDB, MySQL, Kafka with Avro, and Keycloak. Images are built locally for x64 and pushed to a Harbor registry running inside Docker Desktop Kubernetes on a 16 GB Windows laptop. Manifests are kustomize bases and overlays, and Argo CD will manage them later (GitOps). The tight memory budget (a VM of about 8 GB with no swap) shapes many of the decisions.

Layer	Input	Output	Proved by
Code	API contract, domain rules	Tested Maven module	`mvn -pl services/<svc> -am verify`
Image	Jar + Dockerfile	x64 image	`docker build`, `docker run` health check
Registry	Image	Image in Harbor	Harbor UI lists it; a pod can pull it
Manifests	Images + config	Rendered YAML	`kubectl kustomize <dir>`
Runtime	Rendered YAML	Ready pods	`kubectl get pods`, smoke test
GitOps	Git repo	Argo CD Synced/Healthy	Argo CD UI or `kubectl get applications -n argocd`

2. Division of labour: human vs Claude

Claude Code can read and edit files, run shell commands, use MCP tools, and plan and verify its own work. It **cannot** see your screen unless given a tool for it, type passwords for you, sign in to your accounts, change your machine's security settings, or know decisions you never wrote down. It also should not make decisions that only you can be accountable for.

Area	Human (you)	Claude	Shared / how
Goals and scope	Decide what the system does, what *done* means, what is out of scope	Asks clarifying questions, points out gaps	You write it into the checklist/spec; Claude keeps to it
Architecture	Choose services, datastores, sync vs async, ports, names, versions	Proposes options and trade-offs, drafts diagrams	Use plan mode: Claude proposes, you pick
Machine setup	Install tools, size the Docker VM, enable Kubernetes, WSL settings	Checks what's installed, gives exact commands	You run anything that changes system settings
Accounts and secrets	GitHub sign-in, tokens (PAT), Harbor/Keycloak/Argo admin passwords	Never sees or types them; references env vars	`setx GITHUB_PAT ...` done by you; `.env` denied in settings
Code	Defines API contracts and business rules; reviews diffs	Writes code, tests, migrations, Dockerfiles, YAML, scripts	Tests first; you read the test before the code
Verification	Judges whether it is *right* (UX, behaviour, security)	Proves it *works* (runs tests, renders YAML, checks pods)	Gates at the end of each phase
Risky actions	Approves deletes, uninstalls, force-push, deploying extras	Stops and asks; hooks enforce	Permission prompts plus a `guard-bash.sh` hook
Git and GitHub	Owns the remote, merges PRs, branch policy	Branches, commits, pushes, opens PRs	Claude pushes only when you ask
Knowledge	Notices patterns and failures, decides what becomes a rule	Drafts rules, skills, hooks; records memory	After each failure: what rule would have prevented it?

Things only a human can do in this project

Use this as a literal list. Claude can tell you *how* to do each one but should not do it, or cannot.

Machine and environment

- Install Docker Desktop, JDK 21, Maven, Node 20, Git, Helm and kubectl. Claude can produce the `winget` commands and check versions afterwards.
- In Docker Desktop: set VM memory (about 8 GB), enable Kubernetes with cluster type **kubeadm**, and restart Docker or WSL when needed. Changing Docker/WSL settings is on the ask-first list.
- Keep Docker Desktop running whenever tests (Testcontainers) or the cluster are needed.

- Decide the memory budget: what may run at the same time. That is why backend-only is the default.

Accounts, identities and secrets

- Create the GitHub repository and sign Git in to GitHub (Git Credential Manager). In this project's own build session the first `git push` failed with *could not read Username*: Claude cannot sign in for you. You ran the push once, signed in, and from then on Claude could push.
- Create the GitHub fine-grained token for the GitHub MCP server and set it as `GITHUB_PAT` yourself.
- Choose and enter the admin passwords for Harbor, Keycloak and Argo CD, and run `docker login localhost:30002`.
- Approve MCP servers the first time Claude Code asks (`/mcp` shows their status).

Decisions

- Service boundaries, API contracts (endpoints, request/response shapes, status codes), business rules (an order is rejected when stock is short), event schemas (`order-placed.avsc`).
- Versions you commit to: Spring Boot 3.2.x, Java 21, Kafka/Confluent 7.5, MySQL 8.3, Mongo 7.
- Naming and layout: ports, package names, folder structure, image naming.
- When a phase is done, and whether to move to the next one.

Approval and review

- Approve or deny permission prompts. Read the command before you click.
- Review every diff that lands, at least at each gate: tests first, then code, then YAML.
- Use the app in a browser (Swagger UI, frontend, Harbor, Grafana, Argo CD) and judge it.
- Merge PRs. Decide whether to deploy *extras* or *full* (it costs memory).

Learning and capturing knowledge

- Notice when Claude repeats a mistake, or when you repeat an instruction, and turn it into a CLAUDE.md line, a hook or a skill.
- Keep the living docs true: `docs/laptop-setup.md` and your PDF guide.

3. The instruction stack: everything you give Claude

Prompts are only the top layer. Claude Code reads several kinds of instruction, and each has a different lifetime and strength. The most important difference: **CLAUDE.md, skills and docs are advice** that Claude follows from context. **Permissions and hooks are guarantees** that the harness enforces whatever Claude decides.

Component	Lives in	Loaded / active	Strength	Use it for
Prompt	Chat	This turn	Advice	The task: goal, context, constraints, done-criteria
CLAUDE.md	Repo root (+ `CLAUDE.local.md`, `~/.claude/CLAUDE.md`)	Every session	Advice	Facts, commands, always-true rules
Docs / specs	`docs/`	When read or mentioned (`@file`)	Advice	Checklist, API contracts, architecture, troubleshooting
Permissions	`.claude/settings.json` (+ `.local.json`)	Always	**Enforced**	Allowlist safe commands, deny secret files
Hooks	`settings.json` + `.claude/hooks/*.sh`	On events (session start, before/after tools)	**Enforced**	Block dangerous commands, auto-format, validate YAML, inject context
Skills	`.claude/skills/<name>/SKILL.md`	When relevant or `/name`	Advice (procedure)	Repeatable multi-step procedures
Subagents	`.claude/agents/<name>.md`	When delegated	Advice + tool limits	Focused roles in a separate context: reviewer, tester
MCP servers	`.mcp.json`	Session start	Capability	Tools: Kubernetes, GitHub, browsers
Auto memory	`~/.claude/projects/.../memory/`	Index every session	Advice	Personal preferences and lessons across sessions
Modes	Shift+Tab, settings	Per session	**Enforced**	Plan mode (read-only planning), accept-edits, auto

Rule of thumb for choosing the layer

- Something Claude must **know** → CLAUDE.md (short) or a doc (long).
- Something Claude must **do in a fixed sequence** → a skill.
- Something that **must never happen** or must **always** happen → a hook or a permission rule.
- Work that needs a **fresh, focused view** or would flood the main context → a subagent.
- Something Claude needs to **reach** (cluster, GitHub, browser) → an MCP server or CLI tool.

4. Each component in detail

4.1 The prompt

A prompt is a work order. A good one has five parts. You won't need all five every time, but missing ones are where results go wrong.

Part	Question it answers	Example (inventory-service)
Goal	What should exist when you're done?	Create inventory-service with `GET /api/inventory?skuCode=&quantity=` returning true/false.
Context	What does Claude need that isn't in the repo?	It is called by order-service before an order is placed. Seed stock: iphone_15=100, pixel_8=100.
Constraints	What must or must not happen?	MySQL + Flyway migrations, port 8082, package `com.techie.microservices.inventory`, no Lombok in entities.
Done-criteria	How will we prove it?	A Testcontainers MySQL test for both in-stock and out-of-stock cases; `mvn -pl services/inventory-service -am verify` passes.
Boundaries	Where must Claude stop?	Don't touch order-service yet. Show me the test before writing the implementation.

Phrases that steer behaviour well: *plan first and wait for my OK*; *show me the test before the code*; *stop at the gate and report*; *ask before adding any dependency*; *don't run it, I'll review the script*; *explain what you changed in 3 bullets*.

4.2 CLAUDE.md: project memory

- **What:** a Markdown file loaded into every session. Variants: `CLAUDE.md` at the repo root (shared via git), `CLAUDE.local.md` (personal, git-ignored), `~/.claude/CLAUDE.md` (you, across all projects), and CLAUDE.md files in subfolders (picked up when Claude works there).
- **Put in:** facts Claude can't infer (laptop limits, registry URL, ports), the key commands, rules learned the hard way, and the ask-first versus just-do lists.
- **Leave out:** anything the code already shows, long procedures (use skills), and essays. Every line costs context in every session, so aim for under about 100 lines.
- **Create it:** run `/init` in an existing repo (Claude drafts it from the code), or start from `docs/templates/CLAUDE.md` in a new repo. Edit it yourself. It is *your* voice.
- **Keep it alive:** when you correct Claude twice about the same thing, add a line.

```
## Rules that always apply
- **Backend-only by default** ... If < 2 GB available or the API server is slow, do not add
    anything.
- **Images**: always build locally for x64 and push to Harbor. Never use
    `saiupadhyayula007/*` images (arm64-only).
- **Never `kubectl apply` a whole folder with `-f`**; use `-k` on a kustomization or the
    scripts.
- **Ask first**: deleting Kubernetes resources, `helm uninstall`, deleting files, force-
    push, changing Docker/WSL settings.
```

For a from-scratch build, the template adds a **Current phase** line. It keeps Claude on one checklist phase at a time and makes it update the line at each gate. This one line is the cheapest way to stop scope creep.

4.3 Docs and specs

Long, stable knowledge goes in `docs/`, and CLAUDE.md points to it. Claude reads a doc when you mention it (`@docs/build-from-scratch-checklist.md`) or when CLAUDE.md tells it to. For this project:

Doc	Purpose	Who writes
`docs/build-from-scratch-checklist.md`	Phases, steps and gates. The plan Claude follows	Claude drafts, you edit
API contracts (e.g. `docs/api.md` or OpenAPI)	Endpoints, payloads, errors per service. The spec tests are written from	**You** decide; Claude formats
`docs/architecture.png` / Mermaid	Who calls whom, sync vs Kafka	Claude drafts, you confirm
`docs/laptop-setup.md`	Fixes and troubleshooting found along the way	Claude, after each fix
`docs/Laptop-Kubernetes-Guide.pdf`	Your own learning guide	You (Claude says which sections changed)

> **Why API contracts matter most**
>
> A contract you wrote is the strongest spec you can give: Claude writes the tests from it, and the tests decide whether the code is right. Without one, Claude invents a reasonable API. It may differ from what the frontend or order-service expects, and you find out in phase 7.

4.4 Permissions: settings.json

- **What:** `allow` / `ask` / `deny` rules for tools, e.g. `Bash(kubectl get *)` or `Read(./.env)`. The project file `.claude/settings.json` is shared; `.claude/settings.local.json` is yours.
- **Why:** every safe, read-only command you allow is one fewer prompt. Every denied file is a secret Claude never sees. Too many prompts lead to *permission fatigue*, where you click Yes without reading.
- **Start small:** allow builds and read-only inspection. Never allow `kubectl delete`, `helm uninstall` or `git push --force`.

```
{
  "permissions": {
    "allow": ["Bash(mvn *)", "Bash(git status *)", "Bash(git diff *)",
              "Bash(kubectl get *)", "Bash(kubectl describe *)",
              "Bash(kubectl logs *)", "Bash(kubectl kustomize *)"],
    "deny":  ["Read(./.env)", "Read(./**/*.pem)"]
  }
}
```

4.5 Hooks: automatic guarantees

A hook is a script the harness runs at a lifecycle event: `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`, `Stop` and others. A `PreToolUse` hook can **block** a command (exit code 2, with a reason sent back to Claude) or force an *ask*. A `PostToolUse` hook can check what was just written and report problems for Claude to fix. This project has four:

Event	Script	What it guarantees
SessionStart	`session-context.sh`	Every session starts knowing: git state, cluster reachable?, free VM memory, Harbor up?
PreToolUse (Bash)	`guard-bash.sh`	Blocks arm64 images, `-DskipTests`, `kubectl apply -f <folder>`. Asks for deletes, uninstall, extras/full, force-push.
PostToolUse (Edit/Write)	`format-java.sh`	Java is always formatted (Spotless).
PostToolUse (Edit/Write)	`check-k8s.sh`	YAML parses, no arm64/`latest` images, every kustomization renders.

When to write a hook: when a rule in CLAUDE.md was broken once and breaking it is costly. Advice can be forgotten under a long context; a hook cannot. **Test it** without Claude:

```
echo '{"tool_input":{"command":"kubectl delete pod x"}}' | bash .claude/hooks/guard-bash.sh
```

4.6 Skills: reusable procedures

- **What:** a folder `.claude/skills/<name>/` with a `SKILL.md` (frontmatter `name`, `description`, and optionally `argument-hint`). Claude loads it when the description matches the task, or when you type `/name args`.
- **Why:** procedures in CLAUDE.md would cost context every session. A skill loads only when needed, and the steps are the same every time, so service 5 is built like service 1.
- **This project:** `new-service`, `add-feature`, `deploy-local`, `troubleshoot-cluster`, `release-version`.
- **How to get one:** do the task once by hand with Claude, then say: *Turn what we just did into a skill. Include the gate commands and the mistakes we hit.* Edit the result.
- **Write the description carefully.** It decides when the skill triggers: say what it does *and* when to use it.

```
---
name: deploy-local
description: Deploy or start the stack on the laptop's Docker Desktop Kubernetes safely -
    memory check,
  core infrastructure, backend services, health verification and port-forward links. Use
      when the user
  asks to deploy, start, bring up, or run the project locally.
argument-hint: "[full]"
---
1. Preflight: context is docker-desktop; memory `wsl -d docker-desktop free -m`; Harbor ping
    returns Pong.
2. Decide scope: default = core + overlays/local. `full` only if asked AND > 4 GB free.
3. Apply: render first, then `kubectl apply -k ...`, then wait/rollout status.
4. Verify: pods Ready, restarts not climbing, memory re-checked.
5. Hand over the links.
```

4.7 Subagents: focused roles

- **What:** `.claude/agents/<name>.md` with frontmatter `name`, `description`, `tools` and `model`, plus a system prompt. The main Claude delegates a task to it. The subagent works in **its own context window** and returns a report.
- **Why:** (a) a fresh view: a reviewer that didn't write the code finds more; (b) restricted tools: the reviewer is read-only, and `test-engineer` may only touch `src/test`; (c) the main context stays clean.
- **This project:** `code-reviewer`, `test-engineer`, `k8s-manifest-reviewer`.
- **Challenge:** a subagent doesn't see your conversation. Its prompt (written by Claude, or by you) must contain everything it needs: which files, which rules, what to report.

4.8 MCP servers: reach

- **What:** servers that give Claude extra tools, declared in `.mcp.json`: here a read-only `kubernetes` server and the `github` server (issues, PRs). The desktop app also provides a built-in browser.
- **Human jobs:** create tokens, set the env vars, and approve servers on first use. Prefer **read-only** configuration where possible (the Kubernetes MCP here is read-only; changes still go through git and kustomize).
- **Without them:** Claude falls back to CLIs (`kubectl`, `gh`). That works as long as the CLIs are installed and signed in. *gh* was not installed in this project's session, so PR work had to wait.

4.9 Auto memory and modes

- **Memory:** Claude can save small facts across sessions (who you are, preferences, lessons) in a per-project memory folder with an index. Tell it *remember that ...*. Review it occasionally, because stale memories mislead.
- **Plan mode:** Shift+Tab until *plan mode* shows. Claude may only read and think, then presents a plan to approve. Use it at the start of every phase and for anything touching more than a few files.
- **Accept-edits / auto modes:** fewer prompts for routine edits. Use them only once hooks and the deny list protect the dangerous things.

- **Checkpoints:** Esc interrupts Claude. Esc twice or `/rewind` goes back to an earlier point in the conversation and its file edits. Git is still your real safety net, so commit at every gate.

5. Two approaches: workflow vs agentic

You can build the same project in two styles. They are ends of a dial, not two separate tools.

5.1 Workflow (human-driven)

You drive. You open the checklist, take the next step, give Claude one focused prompt, read the diff, run or let it run the gate, commit, and move on. Claude is a fast pair programmer. Instructions live mostly in your prompts and a short CLAUDE.md.

```
You:    "Phase 2, product-service. Plan first: entities, endpoints, test cases. Wait for my
    OK."
Claude: plan
You:    "OK, but use 201 for create. Write the test first and show it."
Claude: ProductServiceApplicationTests.java (Testcontainers Mongo)
You:    "Good. Now implement until mvn -pl services/product-service -am verify passes."
Claude: implements, runs verify, reports
You:    read diff -> commit -> next step
```

Strengths	Challenges
You understand every line, which is best for **learning**	**You are the bottleneck**: progress stops when you stop
Mistakes are caught early, one step at a time	Many permission prompts, which leads to fatigue and rubber-stamping
Little setup: CLAUDE.md + checklist is enough	Prompts vary, so services come out inconsistent
Easy to change direction	Knowledge stays in your head and chat history and is lost between sessions unless written down
Works even when the environment is fragile	Repetitive work (5 services x Dockerfile/YAML/tests) is tedious

5.2 Agentic (Claude drives within guardrails)

You set direction and gates. You invest up front in the instruction stack: a precise CLAUDE.md, specs, skills for each repeatable procedure, subagents for review and tests, hooks that enforce the rules, and permissions that remove safe prompts. Then you give goal-level instructions and Claude runs multi-step work: planning, writing, testing, reviewing with subagents and fixing. It stops at defined gates for your approval.

```
You:    "/new-service inventory-service 8082 MySQL+Flyway. Contract: @docs/api/inventory.md"
Claude: plans -> scaffolds module -> test-engineer writes Testcontainers tests -> implements
        -> mvn verify -> code-reviewer subagent -> fixes findings -> Dockerfile build
        -> k8s manifests (check-k8s hook validates) -> overlay image entry -> stops at gate,
            reports
You:    review summary + diff -> "/deploy-local" -> smoke test in browser -> merge
```

Strengths	Challenges
Speed and consistency: service 5 looks like service 1	**Up-front investment**: skills, agents and hooks take days to get right

Strengths	Challenges
Rules are enforced, not hoped for (hooks)	**Over-trust**: long runs produce big diffs you review less carefully
Built-in review loop (subagents) catches issues before you see them	Harder to follow *why* something was done, so ask for summaries and decisions
Knowledge lives in files, so it survives sessions and is shareable	**Tooling rots**: when ports or layout change, skills and hooks must change too
You spend your time on decisions and review	Environment failures (cluster down, VM out of memory) derail long autonomous runs
Repeatable: the same skill builds the next service	More tokens and cost; subagents need complete briefs; secrets and permissions must be designed

5.3 Side by side

Dimension	Workflow	Agentic
Unit of instruction	One step (*write the test for X*)	One goal (*add inventory-service*)
Where knowledge lives	Prompts, your head	CLAUDE.md, docs, skills, agents, hooks
Human touchpoints	Every step	Gates: plan approval, end of phase, deploy, merge
Safety comes from	You reading each command	Hooks, permissions, plan mode, gates
Best for	Phases 0-1, the first service, anything new to you	Services 2-5, manifests, release loop, repeated fixes
Main risk	Slow; inconsistent	Confident wrong work; big unreviewed diffs
You learn	The technology	How to design instructions and guardrails

5.4 Recommended: the ratchet

For learning Claude Code on this project, use **workflow first, then codify, then go agentic**. Each time something works by hand, lock it in so it can't slip back.

1. **Phases 0-1, workflow.** Write CLAUDE.md (from the template) and a minimal permissions allowlist on day one.
2. **First service (product), workflow, slowly.** Plan mode, test first, read everything. Write down every correction you give.
3. **Codify.** Ask Claude to turn the product-service session into a `new-service` skill. Add a `test-engineer` and a `code-reviewer` subagent. Turn the corrections into CLAUDE.md lines.
4. **Services 2-5, agentic.** `/new-service ...` with your contract doc. You review at the gate. Improve the skill every time it falls short.
5. **First painful mistake → hook.** E.g. an arm64 image or `apply -f k8s/`: add it to `guard-bash.sh`. Add `check-k8s.sh` when you start phase 6.

6. **Deploy and release, agentic with human gates.** `/deploy-local`, `/release-version`, `/troubleshoot-cluster`. You keep memory decisions, merges and anything destructive.

Agentic piece	Build it when	Trigger
CLAUDE.md + permissions	Phase 1 (day one)	Always
Checklist + current-phase line	Phase 1	Always
`new-service` skill	After product-service passes its gate	You are about to repeat it 4 times
`test-engineer`, `code-reviewer` agents	After the first service	You want review you didn't write
`guard-bash.sh` hook	First dangerous near-miss (or before phase 5)	A CLAUDE.md rule was broken
`check-k8s.sh` hook	Start of phase 6	YAML errors are found late
`session-context.sh` hook	Phase 7	Sessions start against a dead cluster
`deploy-local`, `troubleshoot-cluster` skills	After the first manual deploy and first outage	Same diagnosis twice
MCP servers	Phase 7-9	Too much copy-paste of kubectl/GitHub output

6. Phase playbook

Each phase follows the same shape. **Human:** what only you do. **Prompt:** what to give Claude (workflow style, with the agentic version where it differs). **Gate:** proof the phase works; do not move on without it. **Watch out:** what typically goes wrong. Start each phase in plan mode and commit at each gate.

Phase 0: Laptop prerequisites

Human

- Install the tools, set Docker VM memory to about 8 GB, enable Kubernetes (kubeadm).
- Sign Git in to GitHub once (a `git push` or `git credential-manager` login), and install the `gh` CLI if you want Claude to open PRs.
- Install Claude Code and start it in an empty project folder.

Prompt

```
I'm on Windows 11 x64 with 16 GB RAM. Check which of these are installed and their versions:
java, mvn, node, git, gh, helm, kubectl, docker. Also check kubectl context and node status.
Don't install anything or change settings - give me the winget commands for what's missing.
```

Gate

`java -version` shows 21, `kubectl get nodes` shows docker-desktop Ready, `git push` works without a prompt.

Watch out

- Claude must not change Docker/WSL settings. That is yours (and on the ask-first list).
- Git credentials not set up = Claude's pushes fail later (this happened while preparing this guide).

Phase 1: Repository skeleton

Human

- Create the empty GitHub repo. Decide versions, group id and package names.
- Copy `docs/templates/CLAUDE.md` to the root. Set *Current phase: 1*. Read every rule and delete or edit what you don't agree with.
- Write the checklist into `docs/` (or ask Claude to draft it from the tree, then edit).

Prompt

```
Phase 1. Read CLAUDE.md and @docs/build-from-scratch-checklist.md.
Create the parent pom.xml (Spring Boot 3.2.x, Java 21, Testcontainers BOM, Spotless with
    palantir format),
no modules yet, plus .gitignore, .dockerignore and a short README. Show me the pom before
    writing.
Then add .claude/settings.json allowing mvn, git status/diff/log and denying .env.
```

Agentic version

```
/init   (then edit what it wrote)
"Do phase 1 from the checklist. Plan first, then stop at the gate and report."
```

Gate

`mvn -q validate` succeeds; first commit on a branch, pushed.

Watch out

- Versions from memory may be stale or invented. Pin the ones *you* chose, or ask Claude to check Maven Central.
- Claude creates `services/` subfolders early. The phase line should stop that.

Phase 2: Microservices (product → inventory → order → notification → gateway → frontend)

Human

- **Write the API contract and business rules per service** (endpoints, payloads, status codes, events). This is the biggest human input of the whole project.
- Read each test before the implementation: the test *is* your spec in executable form.
- Keep Docker Desktop running (Testcontainers).
- After product-service: ask for the `new-service` skill and the two subagents (the ratchet).

Prompt

```
Phase 2, product-service (port 8080, MongoDB). Contract: @docs/api/product.md
Plan first: package layout, entity, DTOs, endpoints, test cases. Wait for my OK.
Then write the @SpringBootTest + Testcontainers (mongo:7.0.5) + RestAssured test first and
    show it.
Then implement until `mvn -pl services/product-service -am verify` passes. Report what you
    changed.
```

Agentic version

```
/new-service order-service 8081 "MySQL+Flyway, Kafka Avro producer, calls inventory"
Contract: @docs/api/order.md  Event: order-placed.avsc (fields: orderNumber, email,
    firstName, lastName)
Use a Resilience4j circuit breaker + retry on the inventory call; WireMock it in tests.
Run test-engineer and code-reviewer; fix findings; stop at the gate.
```

Gate

`mvn -pl services/<svc> -am verify` passes for every service, and `code-reviewer` reports no blocking findings.

Watch out

- order-service has the most moving parts: Avro classes generated by the plugin, Schema Registry URL, Kafka serializers, circuit-breaker config, WireMock stubs. Do it as a sequence of small prompts, even in agentic mode.
- Tests that mock everything prove nothing. Insist on real containers.

- *Done* without running verify. CLAUDE.md says to run it; check that the output was shown.
- The gateway needs Keycloak, which lives in *extras*. Decide early whether tests use a mocked JWT.

Phase 3: Local dev with docker compose (optional)

Human

- Decide whether you want it (fast feedback without Kubernetes) and accept the memory cost while it runs.

Prompt

```
Create local-dev/docker-compose.yml with mysql (init.sql creating order_service and
    inventory_service DBs),
mongo, kafka (KRaft or zookeeper), schema-registry. Pinned versions, memory limits. Don't
    start it.
```

Gate

`docker compose up -d`, then each service started from the IDE responds on its port.

Watch out

- Running compose *and* the cluster together exhausts the 8 GB VM.

Phase 4: Containerisation

Human

- Choose the base images (e.g. `eclipse-temurin:21-jre`) and confirm they are multi-arch or x64.

Prompt

```
Write services/Dockerfile: multi-stage, --build-arg SERVICE=<svc>, Maven build stage with
    dependency
caching, eclipse-temurin:21-jre runtime, non-root user. Then frontend/Dockerfile (node build
    -> nginx).
Build product-service as localhost:30002/microservices/product-service:0.1.0 and show image
    size.
```

Gate

`docker build` succeeds; `docker run` plus the actuator health endpoint returns UP (with its DB reachable).

Watch out

- Never pull the course's `saiupadhyayula007/*` images (arm64-only, *exec format error*). A hook can block them.
- Unpinned `latest` tags make builds unrepeatable.

Phase 5: Harbor registry

Human

- Choose the Harbor admin password and **enter it yourself**. Run `docker login localhost:30002` yourself.
- Approve the `helm install` (it costs about 1 GB of memory).

Prompt

```
Write k8s/platform/harbor/values-local.yaml for the harbor/harbor Helm chart: NodePort
    30002, no TLS,
externalURL http://localhost:30002, small PVCs, trivy and notary disabled, resource limits.
Write scripts/windows/install-harbor.ps1 and build-and-push.ps1 [-Tag x] [-All|-Only svc].
Don't run them - I'll review first.
```

Gate

`curl http://localhost:30002/api/v2.0/ping` returns Pong; an image pushed with the script appears in the UI; a test pod pulls it.

Watch out

- Memory: Harbor plus core infra is most of the budget. Check `wsl -d docker-desktop free -m`.
- Never let Claude type the admin password. Set it in the values file yourself or via an env var.

Phase 6: Kubernetes manifests

Human

- Decide each pod's resource budget (requests/limits) and the split between core and extras.
- Review the rendered output, not just the bases.

Prompt

```
Phase 6. Create k8s/infrastructure/core (mysql, mongodb, zookeeper, kafka, schema-registry)
    and
k8s/apps/base/backend (common-config ConfigMap + product/order/inventory Deployments and
    Services) with
requests/limits, readiness/liveness on actuator health, and k8s/apps/overlays/local with
    image tags only
in its images: list. Render each with `kubectl kustomize` and fix errors. Don't apply
    anything.
```

Agentic version

```
"Do phase 6. Use k8s-manifest-reviewer when done." (check-k8s.sh validates every edit
    automatically)
```

Gate

`kubectl kustomize` renders every kustomization cleanly, and the reviewer finds no laptop-limit or image-policy problems.

Watch out

- Kafka advertised listeners and service DNS names (`broker:29092`, `schema-registry:8081`) are the classic bug.
- Image tags in bases instead of the overlay break the GitOps model.
- Never `kubectl apply -f k8s/`. Use `-k` on one kustomization.

Phase 7: First deploy (backend only)

Human

- Check memory (or trust the session hook), approve the deploy, and **do the smoke test yourself** in the browser/Swagger: create product → check stock → place order.

Prompt

```
Deploy backend-only: memory check first; stop if < 2 GB available. Apply core, wait for
    Ready,
then overlays/local, wait for rollout. Show pod status and the port-forward commands.
```

Agentic version

```
/deploy-local
```

Gate

All pods Ready with restarts not climbing; the order flow works end to end.

Watch out

- *TLS handshake timeout* from kubectl = VM starved of memory. Scale down; don't add more.
- CrashLoopBackOff → `kubectl logs deploy/<svc> --previous`. Usually DB/Kafka config. Use `/troubleshoot-cluster`.
- Extras / full: only when *you* decide.

Phase 8: GitOps with Argo CD

Human

- Decide to install it (memory). Log in to the Argo CD UI yourself. Connect the repo (public, or a token you enter).
- From now on, resist `kubectl edit`: every change goes through git.

Prompt

```
Phase 8. Create argocd/projects/microservices.yaml, applications/infrastructure.yaml,
applications/microservices-local.yaml and bootstrap/root-app.yaml (app-of-apps) pointing at
    this repo's
main branch. Give me the install commands for Argo CD with reduced resources; don't run
    them.
```

Gate

Root app and children are Synced/Healthy; changing a tag in git and pushing updates the cluster.

Watch out

- Manual changes are reverted by Argo (self-heal) or show as drift. That's the point.
- Argo needs repo credentials if the repo is private.

Phase 9: Release loop

Human

- Decide what to build next. Approve merges and releases.

Prompt

```
Add a feature: GET /api/product/{id} returns 404 for unknown ids. Test first.
```

Agentic version

```
/add-feature product-service "GET /api/product/{id}, 404 when missing"
/release-version product-service 0.2.0      (build, push, bump overlay tag, commit, PR)
```

Gate

Tests green, image in Harbor, tag bumped in `overlays/local`, Argo synced, feature works in the browser.

Watch out

- Never `kubectl set image` as the final state. The tag lives in git.

Phase 10-11: Docs and agentic setup

Human

- Decide which rules become hooks. Create the GitHub token and set `GITHUB_PAT` yourself. Approve MCP servers.
- Update your PDF guide's sections that Claude says changed.

Prompt

```
Update docs/laptop-setup.md with everything we fixed this phase, and tell me which sections
    of
docs/Laptop-Kubernetes-Guide.pdf need updating.
---
Add a PreToolUse hook that blocks `kubectl apply -f <directory>` and `-DskipTests`, and asks
    before
`kubectl delete` or `helm uninstall`. Show me how to test it without running Claude.
```

Gate

Each hook passes a manual test with piped JSON; a fresh session prints the session snapshot.

Watch out

- Hooks on Windows run in Git Bash; `jq` may be missing, so make scripts degrade gracefully.
- When the setup changes, update skills and hooks in the same commit, or they rot.

7. Writing instructions that work

Weak instruction	Why it fails	Strong instruction
Build the order service.	No contract, no done-criteria, unlimited scope	*Create order-service per @docs/api/order.md; plan first; test first; done = verify passes.*
Fix the pod.	Which pod, what symptom, may it delete things?	*order-service is CrashLoopBackOff. Diagnose with logs --previous and describe; propose a fix in YAML; don't delete anything.*
Make it production ready.	Undefined; invites over-engineering	*Add readiness/liveness probes and requests/limits to the three backend Deployments.*
Use best practices.	Claude's defaults may not fit a 16 GB laptop	*Keep each pod under 512 Mi limit; total backend under 3 GB.*
Deploy everything.	Exceeds memory; conflicts with a rule	*Deploy backend-only. If > 4 GB free afterwards, ask me about extras.*
Push it.	Fine, but expect branch rules and credentials	*Commit on a branch, push, and give me the PR link.*

Patterns worth memorising

- **Plan, then act:** *Plan first and wait for my OK.* Use plan mode for anything over a few files.
- **Test first:** *Show me the test before the implementation.* Your review of the test is your spec review.
- **Stop points:** *Stop at the gate and report. Don't run it; I'll review.*
- **Explain:** *Summarise what you changed and why in 5 bullets, including anything you're unsure about.*
- **Reference, don't paste:** `@docs/api/order.md` instead of pasting the contract into chat.
- **Correct once, codify:** *Add that as a rule to CLAUDE.md* or *make that a hook*.
- **Interrupt early:** press Esc as soon as Claude heads the wrong way. Redirecting is cheaper than undoing.
- **Fresh context per phase:** start a new session (or `/clear`) per phase. CLAUDE.md and the checklist carry the state, not chat history.

8. Challenges and how to handle them

Challenge	What it looks like here	Handle it by
Context limits	Long sessions: early rules fade; the conversation gets compacted	One phase per session; state in CLAUDE.md + checklist; hooks for must-never rules
Stale or invented facts	Old library versions, wrong chart values, made-up config keys	Pin versions you chose; ask Claude to verify against docs; the gates catch the rest
Environment, not code	Cluster unreachable, VM out of memory, Docker stopped	Session-start snapshot hook; `troubleshoot-cluster` skill; fix the environment before the code
Credentials	Push fails (no Git login); MCP server fails (bad token)	Human sets up auth once; Claude references env vars only; never paste secrets in chat
False *done*	*Implemented!* but verify never ran, or tests mock everything	Done-criteria in every prompt; the CLAUDE.md test rule; read the verify output
Scope creep	Claude also refactors, adds Lombok, starts the next phase	Current-phase line; *only touch X*; plan mode; ask-before-adding-dependency rule
Permission fatigue	Dozens of prompts and you stop reading them	Allowlist read-only commands; hooks guard the dangerous ones, so prompts that remain mean something
Big diffs	Agentic run changes 40 files	Gates per service; ask for a summary; review tests and YAML first
Platform quirks	Windows: PowerShell vs Git Bash syntax, CRLF, paths with spaces	State the platform in CLAUDE.md; keep scripts in one shell (PowerShell for scripts/windows)
Tooling rot	Port changed, but the skill and hook still use the old one	Change skills/hooks/docs in the same commit; CLAUDE.md rule: update docs when setup changes
Untrusted input	Logs, web pages or files containing *instructions*	Claude treats tool output as data; you stay the only source of instructions

9. Claude Code cheat sheet

Action	How	Use it when
Start in a repo	`claude` in the repo root (or the desktop app's Code tab)	Every session
Generate CLAUDE.md	`/init`	Existing repo without one
Plan mode	Shift+Tab until *plan mode*	Start of each phase; multi-file changes
Reference a file	`@path/to/file`	Specs, contracts, logs
Interrupt / rewind	Esc; Esc Esc or `/rewind`	Claude heads the wrong way
Free up context	`/compact` (summarise) or `/clear` (fresh)	Long sessions, new phase
Run a skill	`/new-service`, `/deploy-local`, ...	Repeatable procedures
Manage agents / hooks / MCP	`/agents`, `/hooks`, `/mcp` (terminal CLI)	Inspect or create config
Permissions	`/permissions` or edit `.claude/settings.json`	Too many prompts; deny secrets
Memory	*remember that ...*; `/memory`	Preferences and lessons
Review code	`/code-review`, `/security-review`	Before committing or merging
Non-interactive	`claude -p "..."`	Scripts, CI

In the desktop app, the terminal-dialog commands (`/permissions`, `/hooks`, `/config`) are not available in the Code tab. Run them from a terminal `claude` session or edit the files directly.

10. Learning path: exercises

Do these in order on a scratch branch. Each one teaches one part of the instruction stack.

1. **Prompt anatomy.** Ask for product-service twice: once as *build product service*, once with all five prompt parts. Compare the diffs.
2. **Plan mode.** Plan inventory-service in plan mode, change two decisions, then approve.
3. **Test-first review.** Have Claude write only the order-service tests. Find one missing case yourself before any code exists.
4. **CLAUDE.md effect.** Remove the x64 image rule, ask for a Kafka manifest, and see which image it picks. Restore the rule.
5. **Write a skill.** Turn your product-service session into `new-service`; use it for notification-service; improve it once.
6. **Write a hook.** Make `guard-bash.sh` block `-DskipTests`. Test it with piped JSON, then watch it stop Claude.
7. **Subagent brief.** Ask `code-reviewer` to review order-service. Then rerun with a vague brief and compare.
8. **Agentic run.** `/new-service` for a small new service (e.g. a *review-service*). Review only at the gate. Count what you had to fix.
9. **Failure drill.** Scale the VM down or deploy extras, then use `/troubleshoot-cluster` and record the fix in `docs/laptop-setup.md`.
10. **Retrospective.** List every correction you gave this week. Each one becomes a CLAUDE.md line, a skill step or a hook. That list is your personal Claude Code method.

Appendix A: Starter files

A.1 CLAUDE.md for a from-scratch build (excerpt)

Full file: `docs/templates/CLAUDE.md`.

```
# Spring microservices on Kubernetes (laptop), built from scratch
## Where we are
- **Current phase: 0** (update this line when a phase's gate passes)
- Work only on the current phase. Do not create files that belong to later phases.
- At the end of a phase: run its gate, tick the boxes in the checklist, update the line
    above, tell me.
## Decisions already made (don't re-ask, don't change without asking)
- Services and ports: product 8080, order 8081, inventory 8082, notification 8083, gateway
    9000
- Registry: Harbor http://localhost:30002, project microservices
## Rules that always apply
- Tests first (Testcontainers), x64 images only, -k never -f, memory check before deploying,
  ask before deletes/uninstall/force-push/Docker settings/new dependencies.
```

A.2 Subagent skeleton

```
---
name: code-reviewer
description: Reviews Java/Spring Boot changes in services/ for bugs, security, resilience
    and test
  coverage. Use proactively after any code change, before committing.
tools: Read, Grep, Glob, Bash
model: sonnet
---
You review changes in services/. Read-only: never edit files.
Check: correctness, error handling, resilience (timeouts, circuit breakers), security (no
    secrets,
input validation), and that every changed behaviour has a Testcontainers integration test.
Report findings ranked by severity with file:line and a concrete fix.
```

A.3 Hook skeleton (PreToolUse, Bash)

```
#!/usr/bin/env bash
# Reads the tool call as JSON on stdin; exit 2 = block (stderr goes back to Claude).
cmd=$(cat | sed -n 's/.*"command":"\([^"]*\)".*/\1/p')
case "$cmd" in
  *saiupadhyayula007/*) echo "Blocked: arm64-only image. Build locally for x64." >&2; exit 2
      ;;
  *-DskipTests*)        echo "Blocked: never skip tests." >&2; exit 2 ;;
esac
exit 0
```

A.4 Phase prompt template

```
Phase <n>: <name>. Read CLAUDE.md and @docs/build-from-scratch-checklist.md (phase <n>).
Goal: <what exists at the end>
Context: <anything not in the repo>
Constraints: <versions, ports, limits, files not to touch>
Done when: <gate command(s)> pass and you've shown me the output.
Plan first and wait for my OK. Stop at the gate and report; update the checklist and the
    phase line.
```

Appendix B: Case study: how this guide's own session went

The conversation that produced this guide is a small model of the whole method:

Step	What happened	Lesson
Ask for structure	Asked for a tree of actions; Claude read the repo first, then produced it	Claude grounds its answer in the code when it can. Ask for structure before details
Persist it	*Save it as a checklist in docs*: tree → `docs/build-from-scratch-checklist.md`	Chat is temporary; files carry state between sessions
Push	Claude committed on a branch (not main), and the push **failed: no Git credentials**	Auth is a human job. Claude reported the failure, didn't work around it, and handed you the command
Verify	*Check if everything is in GitHub*: Claude fetched and compared local and remote commits	Ask for verification, not reassurance. *Done* needs evidence
Codify	Generated a from-scratch CLAUDE.md template with a current-phase line	Turn the plan into durable instructions for future sessions
Learn	*How do I think about this?* → mental models → this guide	Use Claude to explain, not just to type

The same loop applies at every scale: **decide → instruct → let Claude do → verify at a gate → codify what you learned**.